

Essential Regular Expression (Regex) Syntax

1. Basic Characters

- `a`, `b`, `1`, etc.: Matches the exact character.

Example: `abc` matches the string "abc".

2. Metacharacters

- `.`: Matches any single character except a newline.

Example: `a.c` matches "abc", "a1c", etc.

- `^`: Anchors the match to the start of a string or line.

Example: `^abc` matches "abc" at the beginning of a string.

- `$`: Anchors the match to the end of a string or line.

Example: `abc$` matches "abc" at the end of a string.

- `\`: Escapes special characters (e.g., `\.` matches a literal dot).

Example: `\d` matches any digit (0-9).

3. Character Classes

- `[abc]`: Matches any one of `a`, `b`, or `c`.

Example: `[aeiou]` matches any vowel.

- `[^abc]`: Matches any character except `a`, `b`, or `c`.

Example: `[^0-9]` matches any non-digit character.

- `[a-z]`: Matches any lowercase letter from `a` to `z`.

- `[A-Z]`: Matches any uppercase letter from `A` to `Z`.

- `[0-9]`: Matches any digit from `0` to `9`.

4. Predefined Character Classes

- `\d`: Matches any digit (`[0-9]`).

Example: `\d{2}` matches two digits, such as "12".

- `\D`: Matches any non-digit character.

Example: `\D+` matches one or more non-digit characters.

- `\w`: Matches any word character (alphanumeric or underscore).

Example: `\w+` matches "hello_world".

- `\W`: Matches any non-word character.

Example: `\w+` matches punctuation or spaces.

- `\s` : Matches any whitespace character (space, tab, newline).

Example: `\s+` matches one or more whitespace characters.

- `\S` : Matches any non-whitespace character.

5. Quantifiers

- `*` : Matches 0 or more occurrences.

Example: `a*` matches zero or more "a"s.

- `+` : Matches 1 or more occurrences.

Example: `a+` matches one or more "a"s.

- `?` : Matches 0 or 1 occurrence.

Example: `a?` matches zero or one "a".

- `{n}` : Matches exactly `n` occurrences.

Example: `a{3}` matches "aaa".

- `{n,}` : Matches at least `n` occurrences.

Example: `a{2,}` matches "aa", "aaa", etc.

- `{n,m}` : Matches between `n` and `m` occurrences.

Example: `a{1,3}` matches "a", "aa", or "aaa".

6. Grouping and Capturing

- `(abc)` : Capturing group for "abc".

Example: `(abc|def)` matches "abc" or "def".

- `(?:abc)` : Non-capturing group for "abc".

Example: `(?:abc|def)` matches "abc" or "def" without capturing.

- `(?P<name>abc)` : Named capturing group.

Example: `(?P<word>\w+)` captures a word and names it "word".

7. Alternation

- `a|b` : Matches either `a` or `b`.

8. Anchors

- `\b` : Matches a word boundary.

Example: `\bword\b` matches "word" as a whole word.

- `\B` : Matches when not at a word boundary.

Example: `\Bword\b` matches "sword" or "wording", but not "word" alone.

- `^` : Start of line or string.

Example: `^Hello` matches "Hello" at the start of a string.

- `$` : End of line or string.

Example: `world$` matches "world" at the end of a string.

9. Look-Around

- `(?=...)` : Positive lookahead.

Example: `\d(=? dollars)` matches a digit only if followed by " dollars".

- `(?!...)` : Negative lookahead.

Example: `\d(?! dollars)` matches a digit not followed by " dollars".

- `(?<=...)` : Positive lookbehind.

Example: `(?<=\$)\d+` matches digits preceded by a dollar sign.

- `(?<!...)` : Negative lookbehind.

Example: `(?<!\$)\d+` matches digits not preceded by a dollar sign.

Tip: Regex syntax and features can vary between programming languages and tools. Always consult your engine's documentation for full details and support.